

POLITECNICO DI TORINO



Lab 4

Exercise 2 - Netcat

Music and multimedia streaming over the Internet

Academic year 2025/2026

Full Name : MEHRAN BAYAT

Student Number : S320636.

Cristina Rottondi, Leonardo Severi, Massimiliano Zanoni

Version: 2026

©2025-26

README

Lab 4 Exercise 2: Netcat

TCP Communication Using Netcat and Wireshark

Loopback Interface, Localhost Packets, and TCP Flags

Final_Report.md

Contents

1	Title	4
2	Abstract	4
3	Exercise Objective	4
4	Lab Environment and Tools	5
4.1	Working Environment	5
4.2	Wireshark Capture Interface	5
4.3	Capture Noise and Display Filter	5
5	Netcat TCP Execution	5
5.1	Server Command	5
5.2	Client Command	5
5.3	Message Exchange	6
6	Wireshark Observation	6
6.1	Loopback Capture During the Netcat Test	6
6.2	Filtered Packet Sequence	6
7	TCP Connection Initiation	7
7.1	Three-Way Handshake	7
7.2	Meaning of the Ports	7
8	TCP Data Exchange	8
9	TCP Connection Closing	8
9.1	Observed Closing Packets	8
9.2	Why the Final Packet Was RST	8
10	Direct Answers for the Lab Deliverable - TCP	8
10.1	Commands Used	9
10.2	Packets Used to Initiate and Close the TCP Connection	9
10.3	Source Port Client to Server	9
11	UDP Part of the Netcat Exercise	9
11.1	Required UDP Commands	9
11.2	Main Packet Difference Between TCP and UDP	9

11.3 Differences Between TCP and UDP Packets	10
11.4 Strange Behavior in UDP Packet Order	10
11.5 Important Note About My UDP Section	10
12 Comparison Summary	11
13 Final Summary	11
References	11

1. Title

Netcat TCP Communication Analysis Using Terminal and Wireshark in the Crownlabs Lab 2 Virtual Machine

2. Abstract

In this report I explain what I did for **Lab 4 - Exercise 2: Netcat**. In this exercise I used Netcat to create a simple local communication between a server and a client, and then I checked the packets with Wireshark. The main goal was to see how TCP works in practice and how its connection setup, data exchange, and closing packets appear in a real capture.

For my practical capture, I completed the **TCP part**. I started a Netcat server on 127.0.0.1 and port 8080. Then I opened another terminal and connected a Netcat client to the same address and port. Since both the server and the client were running inside the same VM, I captured the traffic on **Loopback: lo**. During the first capture, I also saw many VNC packets from the Crownlabs remote desktop, so I used the display filter `tcp.port == 8080` to focus only on my Netcat packets.

The important result is that TCP did not send only the text message. Before sending data, TCP first opened the connection with the three-way handshake. In my capture, the client source port was **55760** and the server port was **8080**. The connection started with **SYN, SYN, ACK**, and **ACK**. After that, the data exchange appeared with **PSH, ACK**. At the end, the connection was closed with **FIN, ACK, ACK**, another **FIN, ACK**, and finally a **RST**, because I stopped the Netcat terminals manually.

3. Exercise Objective

The objective of this exercise was to observe transport-layer communication using Netcat and Wireshark. In the TCP part, I had to run Netcat as a server, run another Netcat command as a client, exchange simple text, close the client first, and then identify the TCP packets used to start and close the connection.

The practical workflow that I followed was:

```
Start Wireshark on Loopback -> run TCP Netcat server on 127.0.0.1:8080 -> run TCP
Netcat client -> exchange a message -> close the client first -> inspect packets
with tcp.port == 8080
```

The lab deliverable asks for these points in the TCP part:

1. the command used for the Netcat server;
2. the command used for the Netcat client;
3. the packets used to initiate the TCP connection;
4. the packets used to close the TCP connection;
5. the source port of the packets going from client to server.

The lab also includes a UDP part. I included the required UDP commands and the protocol comparison, but I do not claim exact UDP packet numbers because the exact Wireshark capture we recorded and analyzed here was the TCP capture.

4. Lab Environment and Tools

4.1 Working Environment

I performed this experiment inside the **Crownlabs Lab 2 virtual machine**. I used Terminal for the Netcat commands and Wireshark for the packet capture.

Tool	How I used it
Terminal	I used it to run the Netcat server and client commands.
Netcat	I used it to create the local TCP communication between two terminals.
Wireshark	I used it to capture and inspect the packets on the loop-back interface.

4.2 Wireshark Capture Interface

For this exercise I used **Loopback: lo**. This was the correct interface because both sides of the connection used 127.0.0.1. The packets did not go outside the VM, so **enp1s0** was not the correct interface for this specific experiment.

4.3 Capture Noise and Display Filter

While capturing on loopback, I noticed many unrelated packets. These packets were mainly VNC traffic from the Crownlabs remote desktop connection, for example traffic related to port 5901. These packets were not part of my Netcat communication.

To isolate the Netcat traffic, I used this Wireshark display filter:

```
tcp.port == 8080
```

This filter shows packets where either the source port or the destination port is 8080. This is useful because my Netcat server was listening on port 8080.

5. Netcat TCP Execution

5.1 Server Command

In the first terminal, I started Netcat in listen mode. This terminal acted as the server:

```
nc -l 127.0.0.1 8080
```

After I ran this command, the terminal looked empty. This was normal because the server was waiting for a client connection.

5.2 Client Command

In the second terminal, I connected to the Netcat server with this command:

```
nc 127.0.0.1 8080
```

After this command, the client connected to the server and the TCP handshake was created.

5.3 Message Exchange

After the connection was established, I sent a simple text message between the two terminals. This was enough to generate data packets in Wireshark. In the packet list, the data exchange appeared as a TCP packet with PSH, ACK. This means that data was pushed through the TCP connection, not only control packets.

6. Wireshark Observation

6.1 Loopback Capture During the Netcat Test

Figure 1 shows the Wireshark capture during the Netcat experiment. In the screenshot, Wireshark is capturing on Loopback: lo. The screenshot also shows why filtering was necessary: the loopback interface also contains VNC packets from the remote desktop connection, which are not part of the Netcat test.

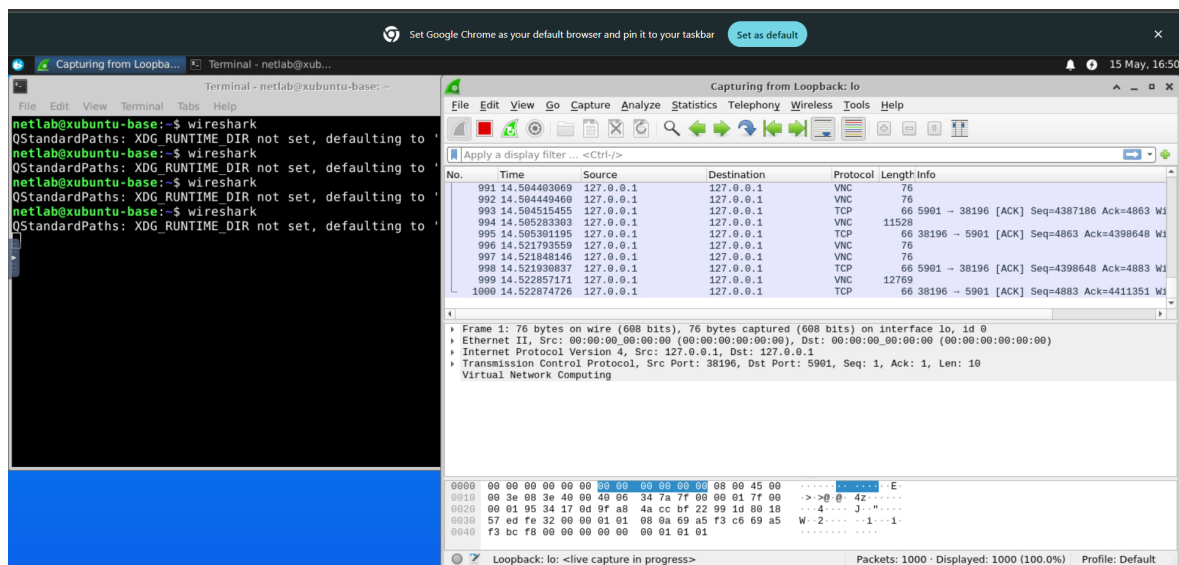


Figure 1: Wireshark capturing on Loopback: lo during the Netcat experiment. The capture contains local packets and also unrelated VNC noise, so I used `tcp.port == 8080` to isolate the Netcat TCP packets.

6.2 Filtered Packet Sequence

After the capture, I filtered the packets with `tcp.port == 8080`. The important TCP packets from my capture were these:

No.	Source	Destination	Direction	Information
11284	127.0.0.1	127.0.0.1	55760 → 8080	[SYN] - the client asks to start the connection.
11285	127.0.0.1	127.0.0.1	8080 → 55760	[SYN, ACK] - the server accepts and acknowledges the request.
11286	127.0.0.1	127.0.0.1	55760 → 8080	[ACK] - the client completes the handshake.
13404	127.0.0.1	127.0.0.1	8080 → 55760	[PSH, ACK] - a data packet carrying the Netcat text.
13405	127.0.0.1	127.0.0.1	55760 → 8080	[ACK] - the client acknowledges the data.
14150	127.0.0.1	127.0.0.1	8080 → 55760	[FIN, ACK] - one side starts closing the connection.
14157	127.0.0.1	127.0.0.1	55760 → 8080	[ACK] - the close request is acknowledged.
18260	127.0.0.1	127.0.0.1	55760 → 8080	[FIN, ACK] - the client side sends its own close request.
18261	127.0.0.1	127.0.0.1	8080 → 55760	[RST] - the connection is reset after manual interruption.

7. TCP Connection Initiation

7.1 Three-Way Handshake

The TCP connection started with the standard three-way handshake:

```
55760 -> 8080 [SYN]
8080 -> 55760 [SYN, ACK]
55760 -> 8080 [ACK]
```

This is the normal way TCP starts a connection. The client first sends **SYN**. Then the server replies with **SYN, ACK**. Finally, the client sends **ACK**. After this, both sides can exchange data.

7.2 Meaning of the Ports

The server port was 8080 because I selected it in the Netcat server command. The client port was 55760. I did not choose this port manually; it was automatically assigned by the operating system as a temporary client port.

Port	Meaning
8080	Fixed server listening port used by the Netcat server.
55760	Temporary source port used by the Netcat client.

So the source port in the direction client → server was:


```
55760
```

8. TCP Data Exchange

After the handshake, I observed a data packet:

```
8080 -> 55760 [PSH, ACK]
```

The **PSH** flag shows that TCP was carrying application data. In this case, that data came from the text typed in the Netcat terminal. The next **ACK** confirmed that the other side received the data.

This shows that TCP does not work like a single independent message. It first creates a connection, then transfers data as a reliable byte stream using sequence and acknowledgment numbers.

9. TCP Connection Closing

9.1 Observed Closing Packets

The connection closing part in my capture was:

```
8080 -> 55760 [FIN, ACK]
55760 -> 8080 [ACK]
55760 -> 8080 [FIN, ACK]
8080 -> 55760 [RST]
```

The first **FIN, ACK** means that one side wanted to close its sending direction. The other side answered with **ACK**. Then the client also sent its own **FIN, ACK**. Instead of a final clean **ACK**, my capture ended with a **RST** packet from port 8080.

9.2 Why the Final Packet Was RST

In a fully graceful TCP close, the last packet is usually an **ACK**. In my capture, the final packet was **RST**. This happened because I stopped the Netcat terminals manually using **Ctrl + C**. For this reason, the application or operating system reset the connection instead of finishing with the clean final acknowledgment.

This does not change the main result of the exercise. TCP still clearly showed connection setup packets, data exchange, and connection closing packets. This is exactly the behavior that makes TCP different from UDP.

10. Direct Answers for the Lab Deliverable - TCP

This section contains the direct answers for **4.2.1 TCP**.

10.1 Commands Used

1. **Server:** `nc -l 127.0.0.1 8080`
2. **Client:** `nc 127.0.0.1 8080`

10.2 Packets Used to Initiate and Close the TCP Connection

Connection initiation packets:

```
55760 -> 8080 [SYN]
8080 -> 55760 [SYN, ACK]
55760 -> 8080 [ACK]
```

Connection closing packets observed:

```
8080 -> 55760 [FIN, ACK]
55760 -> 8080 [ACK]
55760 -> 8080 [FIN, ACK]
8080 -> 55760 [RST]
```

10.3 Source Port Client to Server

The source port of the packets going from the client to the server was:

```
55760
```

I read this value from the first handshake packet, where Wireshark showed:

```
55760 -> 8080 [SYN]
```

11. UDP Part of the Netcat Exercise

11.1 Required UDP Commands

For the UDP version, the same idea is repeated but with the `-u` option. The commands are:

```
Server:
nc -u -l 127.0.0.1 8080

Client:
nc -u 127.0.0.1 8080
```

The `-u` option changes Netcat from TCP mode to UDP mode.

11.2 Main Packet Difference Between TCP and UDP

The main difference is that UDP does not open a connection. Therefore, UDP does not have the TCP control packets used for opening and closing a connection, such as:

```

SYN
SYN, ACK
ACK handshake
FIN, ACK
RST

```

In the TCP capture, several packets were used only for the connection setup and the connection closing. With UDP, the communication mainly appears as UDP datagrams. So for a short message exchange, UDP normally produces fewer packets because it does not need a handshake and does not need a formal closing sequence.

11.3 Differences Between TCP and UDP Packets

Aspect	TCP	UDP
Connection behavior	Connection-oriented. It needs a handshake before data exchange.	Connectionless. It sends datagrams without creating a connection.
Reliability	Uses acknowledgments, sequence numbers, and retransmission support.	Does not guarantee delivery, ordering, or retransmission.
Header complexity	More complex header with flags, sequence number, acknowledgment number, and window field.	Simpler header with source port, destination port, length, and checksum.
Typical packets	SYN, SYN, ACK, ACK, PSH, ACK, FIN, ACK, RST.	UDP datagrams without handshake or closing flags.
Main behavior in this lab	TCP showed a complete connection life cycle.	UDP would show message datagrams without a connection life cycle.

The reason is that TCP is designed for reliable and ordered delivery, while UDP is designed to be simpler and lighter. UDP does not manage the same connection state that TCP manages.

11.4 Strange Behavior in UDP Packet Order

A strange behavior can happen in the UDP version when trying to send the first message from the server/listener side. Since UDP does not create a real connection, the listener may not know the remote client address until the client sends a first datagram. Because of this, the apparent packet order in Wireshark can look different from what I expect from TCP.

In TCP, after the client connects, both sides are part of the same established connection. In UDP, every message is an independent datagram. This is why the packet order and behavior can look less strict than TCP.

11.5 Important Note About My UDP Section

In this report, I documented the TCP capture with exact packet numbers and exact port values because that is the capture I performed and analyzed. For UDP, I included the commands and the required comparison, but I did not invent exact packet numbers from a capture that I did not record here.

12. Comparison Summary

Feature	TCP Netcat	UDP Netcat
Command option	Default Netcat mode, no <code>-u</code> .	Uses <code>-u</code> .
Connection setup	Present: <code>SYN, SYN, ACK, ACK</code> .	Missing: no handshake.
Data transfer	Data appears as TCP segment, for example <code>PSH, ACK</code> .	Data appears as UDP datagrams.
Connection closing	Present: <code>FIN, ACK, ACK</code> , and in my capture final <code>RST</code> .	Missing: no formal close.
Client source port	55760 in my TCP capture.	It would be an automatically assigned UDP client port.
Reliability	Reliable byte stream with sequence and acknowledgment numbers.	Best-effort datagrams without built-in delivery guarantee.

13. Final Summary

For this exercise, I used Netcat and Wireshark to observe a local TCP connection. I captured on Loopback: `lo` because both the client and the server used `127.0.0.1`. I started the server with `nc -l 127.0.0.1 8080`, connected with the client command `nc 127.0.0.1 8080`, exchanged a simple message, and then inspected the packets in Wireshark.

The final result of my TCP capture was:

Item	Final result
Exercise	Lab 4 - Exercise 2: Netcat, TCP part
Interface	Loopback: <code>lo</code>
Server command	<code>nc -l 127.0.0.1 8080</code>
Client command	<code>nc 127.0.0.1 8080</code>
Server port	8080
Client source port	55760
Initiation packets	<code>SYN, SYN, ACK, ACK</code>
Data packet	<code>PSH, ACK</code>
Closing packets observed	<code>FIN, ACK, ACK, FIN, ACK, RST</code>
Display filter used	<code>tcp.port == 8080</code>

This exercise showed me clearly that TCP is connection-oriented. Before the message was exchanged, TCP first created a connection with the three-way handshake. After the data exchange, it used explicit closing packets. This is the main practical difference from UDP, where there is no handshake and no formal closing process.

References

1. Lab 4 assignment PDF, Music and multimedia streaming over the Internet, Politecnico di Torino, Academic year 2025/2026.

2. Lab 4 deliverable template, Section 4.2, Netcat.
3. Wireshark packet capture performed in the Crownlabs Lab 2 virtual machine.